

Software Defined Mobile Networks

Assignment 1

Electrical Engineering Department

Sharif University of Technology

Student Name: Iman Alizadeh Fakouri

Student Number: 401102134

Instructors:

Dr. B. Khalaj
Dr. A. Ravanshid

July 18, 2026

Contents

1	Problem 1: Mininet and OVS	2
1.1	Part 1: Using the OVS Switch	2
1.1.1	Codes and Implementation	2
1.1.2	Results and Command Execution	3
1.2	Part 2: Using Mininet and Implementing a Router	5
1.2.1	Codes and Implementation	5
1.2.2	Results and Command Execution	7
2	Problem 2: Opendaylight	9
2.1	Part 1: ODL intro	9
2.1.1	Codes and Implementation	9
2.1.2	Results and Command Execution	10
2.2	Part 2: Using RESTCONF	12
2.2.1	Codes and Implementation	12
2.2.2	Flow Configurations and Analysis	16
2.2.3	Results and Command Execution	17
3	Problem 3: Minimum Weight Routing	19
3.1	Codes and Implementation	19
3.2	Flow Configurations and Analysis	23
3.3	Results and Command Execution	24
4	Problem 4: Dynamic Routing (Bonus)	25
4.1	Codes and Implementation	25
4.2	Flow Configurations and Analysis	31
4.3	Results and Command Execution	32

1 Problem 1: Mininet and OVS

1.1 Part 1: Using the OVS Switch

In this section, we implement a simple Local Area Network (LAN) topology utilizing Mininet and the Open vSwitch (OVS). The objective is to manually program the switch to forward packets between two hosts without the reliance on an external SDN controller.

1.1.1 Codes and Implementation

```
1 from mininet.net import Mininet
2 from mininet.node import OVSKernelSwitch
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel
5 from mininet.link import TCLink
6
7 def topology():
8     net = Mininet(
9         controller=None,
10        switch=OVSKernelSwitch,
11        link=TCLink
12    )
13
14    h1 = net.addHost('h1', ip='10.0.0.1/24', mac='00:00:00:00:00:01')
15    h2 = net.addHost('h2', ip='10.0.0.2/24', mac='00:00:00:00:00:02')
16    s1 = net.addSwitch('s1', protocols="OpenFlow13")
17    net.addLink(h1, s1, port1=1, port2=1)
18    net.addLink(h2, s1, port1=1, port2=2)
19    net.build()
20    net.start()
21    CLI(net)
22    net.stop()
23
24 if __name__ == '__main__':
25     setLogLevel('info')
26     topology()
```

Listing 1: create_net.py: Network Topology Definition

- The Mininet object is initialized. Crucially, the controller argument is set to None, ensuring the switch operates in a "dumb" state initially, waiting for manual flow rules.
- It explicitly uses the OVSKernelSwitch for efficient packet processing.
- Two virtual hosts (h1 and h2) are created within the 10.0.0.0/24 subnet, and assigned static MAC addresses.
- A single OpenFlow 1.3 compatible switch (s1) is added to the network.
- Links are defined with explicit port mappings. h1 connects to s1 on switch port 1, while h2 connects on switch port 2.
- The network is built and started, and control is handed over to the Mininet Command Line Interface (CLI) for user interaction.

```

1  #!/bin/bash
2
3  echo "--- Pushing flows to s1 ---"
4  sudo ovs-ofctl -O OpenFlow13 del-flows s1
5  sudo ovs-ofctl -O OpenFlow13 add-flow s1 in_port=1,actions=output:2
6  sudo ovs-ofctl -O OpenFlow13 add-flow s1 in_port=2,actions=output:1
7  echo "--- Flows successfully pushed! ---"
8  echo "Current flows in s1:"
9  sudo ovs-ofctl -O OpenFlow13 dump-flows s1

```

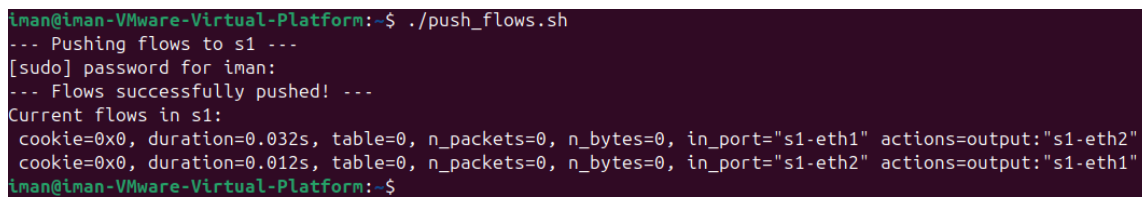
Listing 2: push_flows.sh: Manual Flow Injection

- `del-flows s1` clears any pre-existing or default routing rules from the switch, ensuring a clean slate.
- The `add-flow` commands define the core routing logic and instruct the switch to match traffic entering port 1 (`in_port=1`) and output it to port 2 (`actions=output:2`), and vice versa.
- This establishes bidirectional communication.
- `dump-flows s1` queries the switch and prints the active flow table to the terminal, allowing us to verify that the injection was successful.

1.1.2 Results and Command Execution

To bring the environment online and test it, several Linux commands were utilized across two terminal sessions:

1. `sudo python3 create_net.py`: Executed with superuser privileges, this instantiates the Mininet topology and opens the `mininet>` prompt.
2. `chmod +x push_flows.sh`: Makes the bash script executable.
3. `./push_flows.sh`: Executes the flow-pushing script in a separate terminal while Mininet runs in the background.
4. `pingall` and `h1 ping -c 4 h2`: Mininet CLI commands used to generate ICMP traffic and test network reachability.



```

iman@iman-VMware-Virtual-Platform:~$ ./push_flows.sh
--- Pushing flows to s1 ---
[sudo] password for iman:
--- Flows successfully pushed! ---
Current flows in s1:
 cookie=0x0, duration=0.032s, table=0, n_packets=0, n_bytes=0, in_port="s1-eth1" actions=output:"s1-eth2"
 cookie=0x0, duration=0.012s, table=0, n_packets=0, n_bytes=0, in_port="s1-eth2" actions=output:"s1-eth1"
iman@iman-VMware-Virtual-Platform:~$

```

Figure 1: Execution of the flow-pushing shell script.

This image displays the terminal output of `./push_flows.sh`. It confirms that the flow entries were successfully written to the OVS flow table. The `dump-flows` command output validates that two specific rules exist in `table=0`: one matching `in_port="s1-eth1"` pointing to `actions=output:"s1-eth2"`, and its exact inverse. The absence of error messages indicates successful communication with the OVS kernel module.

```

iman@iman-VMware-Virtual-Platform:~$ sudo python3 create_net.py
[sudo] password for iman:
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>

```

Figure 2: Mininet CLI executing a global ping test.

Here, the `pingall` command is executed inside the Mininet CLI. The output explicitly states `h1 -> h2` and `h2 -> h1` are reachable, resulting in 0% dropped (2/2 received). This justifies that our statically injected flows are functioning correctly, successfully acting as the localized "brain" of the switch to route ICMP packets between the isolated hosts.

```

iman@iman-VMware-Virtual-Platform:~$ sudo python3 create_net.py
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.921 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.084 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3080ms
rtt min/avg/max/mdev = 0.080/0.291/0.921/0.363 ms
mininet> h2 ping -c 4 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.560 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.079 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.080 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.081 ms

--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3061ms
rtt min/avg/max/mdev = 0.079/0.200/0.560/0.207 ms
mininet>

```

Figure 3: Targeted ICMP ping tests between hosts.

To further verify network stability, specific 4-packet ping requests (`-c 4`) were sent from `h1` to `h2`, and from `h2` to `h1`. The terminal logs show standard ICMP echo replies with a 0% packet loss rate and highly stable, sub-millisecond Round Trip Times (RTT). This confirms continuous, stable, and completely bidirectional Layer 2 connectivity facilitated entirely by our custom OpenFlow instructions.

1.2 Part 2: Using Mininet and Implementing a Router

In this section, we expand the topology to include hosts located in different subnets. Because the hosts are no longer in the same broadcast domain, simple Layer 2 forwarding is insufficient. We must implement a virtual router using an OVS switch that reads Layer 3 headers, modifies MAC addresses, and decrements the Time-To-Live (TTL) field to correctly route packets between the subnets.

1.2.1 Codes and Implementation

```
1  from mininet.net import Mininet
2  from mininet.node import OVSKernelSwitch
3  from mininet.cli import CLI
4  from mininet.log import setLogLevel
5  from mininet.link import TCLink
6
7  def topology():
8      net = Mininet(
9          controller=None,
10         switch=OVSKernelSwitch,
11         link=TCLink
12     )
13     h1 = net.addHost('h1', ip='10.0.1.1/24', defaultRoute='via
14         10.0.1.254', mac='00:00:00:00:01:01')
15     h2 = net.addHost('h2', ip='10.0.2.1/24', defaultRoute='via
16         10.0.2.254', mac='00:00:00:00:02:01')
17     s1 = net.addSwitch('s1', protocols="OpenFlow13")
18     s2 = net.addSwitch('s2', protocols="OpenFlow13")
19     r1 = net.addSwitch('r1', protocols="OpenFlow13")
20
21     net.addLink(h1, s1, port1=0, port2=1)
22     net.addLink(s1, r1, port1=2, port2=1)
23     net.addLink(r1, s2, port1=2, port2=1)
24     net.addLink(s2, h2, port1=2, port2=0)
25
26     net.build()
27     net.start()
28     h1.cmd('arp -s 10.0.1.254 00:00:00:00:01:FE')
29     h2.cmd('arp -s 10.0.2.254 00:00:00:00:02:FE')
30
31     CLI(net)
32     net.stop()
33
34 if __name__ == '__main__':
35     setLogLevel('info')
36     topology()
```

Listing 3: create_router_net.py: L3 Network Topology Definition

- The Mininet object is initialized without a default controller (controller=None) and utilizes the OVSKernelSwitch for data plane operations.
- Two hosts are created in distinct subnets (10.0.1.0/24 and 10.0.2.0/24) and are configured with default routes pointing to their respective gateway IP addresses (10.0.1.254 and 10.0.2.254).
- Three OpenFlow 1.3 compatible switches are instantiated: s1 and s2 act as Layer 2 edge switches, while r1 acts as the central Layer 3 router.

- Physical links are explicitly mapped between the hosts, the edge switches, and the central router to establish the required linear topology.
- Static ARP entries are injected into h1 and h2 using the `cmd` function to map the gateway IP addresses to specific virtual MAC addresses (00:00:00:00:01:FE and 00:00:00:00:02:FE), bypassing ARP resolution across subnets.

```

1  #!/bin/bash
2
3  echo "--- Pushing simple L2 flows for Switches (s1, s2) ---"
4
5  sudo ovs-ofctl -O OpenFlow13 del-flows s1
6  sudo ovs-ofctl -O OpenFlow13 del-flows s2
7
8  sudo ovs-ofctl -O OpenFlow13 add-flow s1 in_port=1,actions=output:2
9  sudo ovs-ofctl -O OpenFlow13 add-flow s1 in_port=2,actions=output:1
10
11 sudo ovs-ofctl -O OpenFlow13 add-flow s2 in_port=1,actions=output:2
12 sudo ovs-ofctl -O OpenFlow13 add-flow s2 in_port=2,actions=output:1
13
14 echo "--- Pushing L3 routing flows for Router (r1) ---"
15 sudo ovs-ofctl -O OpenFlow13 del-flows r1
16
17
18 sudo ovs-ofctl -O OpenFlow13 add-flow r1
19   priority=100,ip,nw_dst=10.0.2.1,actions=mod_dl_src:00:00:00:00:02:FE,mod_dl_dst:
20 sudo ovs-ofctl -O OpenFlow13 add-flow r1
   priority=100,ip,nw_dst=10.0.1.1,actions=mod_dl_src:00:00:00:00:01:FE,mod_dl_dst:
20 echo "--- Flows successfully pushed! ---"

```

Listing 4: `push_router_flows.sh`: L2 and L3 Flow Injection

- The script first clears any existing flow rules from the edge switches `s1` and `s2`.
- Simple bidirectional Layer 2 forwarding rules are applied to `s1` and `s2`, instructing them to blindly pass traffic between ports 1 and 2.
- Existing flows on the central router `r1` are deleted before applying complex Layer 3 routing logic.
- OpenFlow rules are added to `r1` matching specific IPv4 destination addresses (`nw_dst=10.0.2.1` and `nw_dst=10.0.1.1`).
- The router actions rewrite the Data Link layer headers by modifying the source MAC (`mod_dl_src`) to the router's virtual MAC, and the destination MAC (`mod_dl_dst`) to the target host's MAC.
- The `dec_ttl` action is explicitly called to decrement the IP Time-To-Live field, which is required to accurately simulate a Layer 3 routing hop before outputting the packet.

1.2.2 Results and Command Execution

To bring the environment online and test it, several Linux commands were utilized across two terminal sessions:

1. `sudo python3 create_router_net.py`: Executed with superuser privileges, this instantiates the Mininet topology and opens the `mininet>` prompt.
2. `chmod +x push_router_flows.sh`: Makes the bash script executable.
3. `./push_router_flows.sh`: Executes the flow-pushing script in a separate terminal while Mininet runs in the background.
4. `pingall`: Mininet CLI command used to generate ICMP traffic and test global network reachability across subnets.
5. `h1 ping -c 4 h2`: Specific Mininet CLI command to trace packet delivery from subnet 1 to subnet 2.

```
iman@iman-VMware-Virtual-Platform:~$ ./push_router_flows.sh
--- Pushing simple L2 flows for Switches (s1, s2) ---
--- Pushing L3 routing flows for Router (r1) ---
--- Flows successfully pushed! ---
iman@iman-VMware-Virtual-Platform:~$
```

Figure 4: Execution of the router flow-pushing shell script.

The terminal output demonstrates the successful execution of the `./push_router_flows.sh` command. The script smoothly executes without throwing any Open vSwitch syntax or kernel errors, proving that the underlying switches properly parsed and applied both the basic Layer 2 forwarding instructions and the complex OpenFlow 1.3 Layer 3 MAC-rewriting actions.

```
iman@iman-VMware-Virtual-Platform:~$ sudo python3 create_router_net.py
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 3 switches
s1 s2 r1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Figure 5: Mininet CLI executing a global ping test across subnets.

The `pingall` command execution confirms that `h1` and `h2` can successfully communicate, showing a 0% drop rate (2/2 received). This verifies that the isolated subnets have been successfully bridged. The OpenFlow instructions injected into `r1` are actively intercepting packets, decrementing the TTL, rewriting the MAC addresses, and successfully forwarding them to the correct subnet destinations.

```

lman@lman-VMware-Virtual-Platform:~$ sudo python3 create_router_net.py
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 3 switches
s1 s2 r1 ...
*** Starting CLI:
mininet> h1 ping -c 4 h2
PING 10.0.2.1 (10.0.2.1) 56(84) bytes of data.
64 bytes from 10.0.2.1: icmp_seq=1 ttl=63 time=0.660 ms
64 bytes from 10.0.2.1: icmp_seq=2 ttl=63 time=0.118 ms
64 bytes from 10.0.2.1: icmp_seq=3 ttl=63 time=0.082 ms
64 bytes from 10.0.2.1: icmp_seq=4 ttl=63 time=0.094 ms

--- 10.0.2.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3049ms
rtt min/avg/max/mdev = 0.082/0.238/0.660/0.243 ms
mininet> h1 ping -c 4 r1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.055 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.043 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.051 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.034/0.045/0.055/0.008 ms
mininet>

```

Figure 6: Targeted ICMP ping tests validating Layer 3 hop mechanics.

A targeted 4-packet ping request from h1 to h2 (10.0.2.1) succeeds with 0% packet loss. Crucially, the ICMP replies report a `ttl=63`. Since standard packets originate with a TTL of 64, this explicitly confirms that the `dec_ttl` OpenFlow instruction on r1 is functioning exactly as intended, legitimately acting as a Layer 3 routing hop within the virtual network.

2 Problem 2: Opendaylight

2.1 Part 1: ODL intro

In this section, we introduce OpenDaylight (ODL) Nitrogen as our SDN controller to automate the routing of a 3-level tree topology. By utilizing ODL's L2 learning switch application and DLUX GUI, we can visualize the topology and establish full network connectivity without manual flow injection.

2.1.1 Codes and Implementation

```
1 from mininet.net import Mininet
2 from mininet.node import RemoteController, OVSKernelSwitch
3 from mininet.cli import CLI
4 from mininet.log import setLogLevel
5 from mininet.link import TCLink
6
7 def tree_topology():
8     net = Mininet(
9         controller=RemoteController,
10        switch=OVSKernelSwitch,
11        link=TCLink,
12        autoSetMacs=True
13    )
14
15    c0 = net.addController('c0', controller=RemoteController,
16        ip='127.0.0.1', port=6633)
17    s1 = net.addSwitch('s1', protocols="OpenFlow13")
18
19    s2 = net.addSwitch('s2', protocols="OpenFlow13")
20    s3 = net.addSwitch('s3', protocols="OpenFlow13")
21
22    s4 = net.addSwitch('s4', protocols="OpenFlow13")
23    s5 = net.addSwitch('s5', protocols="OpenFlow13")
24    s6 = net.addSwitch('s6', protocols="OpenFlow13")
25    s7 = net.addSwitch('s7', protocols="OpenFlow13")
26
27    h1, h2 = net.addHost('h1'), net.addHost('h2')
28    h3, h4 = net.addHost('h3'), net.addHost('h4')
29    h5, h6 = net.addHost('h5'), net.addHost('h6')
30    h7, h8 = net.addHost('h7'), net.addHost('h8')
31
32    net.addLink(s1, s2)
33    net.addLink(s1, s3)
34
35    net.addLink(s2, s4)
36    net.addLink(s2, s5)
37    net.addLink(s3, s6)
38    net.addLink(s3, s7)
39
40    net.addLink(h1, s4); net.addLink(h2, s4)
41    net.addLink(h3, s5); net.addLink(h4, s5)
42    net.addLink(h5, s6); net.addLink(h6, s6)
43    net.addLink(h7, s7); net.addLink(h8, s7)
44
45    net.build()
46    c0.start()
47    for switch in [s1, s2, s3, s4, s5, s6, s7]:
48        switch.start([c0])
```

```

49     CLI(net)
50     net.stop()
51
52 if __name__ == '__main__':
53     setLogLevel('info')
54     tree_topology()

```

Listing 5: create_tree_net.py: 3-Level Tree Topology Definition

- The Mininet environment is initialized with an external RemoteController and the autoSetMacs parameter enabled to simplify host MAC address assignments for cleaner controller logs.
- A remote controller node c0 is instantiated and pointed to the local loopback address on port 6633, instructing Mininet to connect to the external OpenDaylight instance.
- Seven Open vSwitch nodes (s1 through s7) are created, all explicitly configured to use the OpenFlow 1.3 protocol for optimal compatibility with OpenDaylight Nitrogen.
- Eight virtual hosts (h1 through h8) are generated.
- Network links are sequentially added to form a strict three-level tree topology, with s1 acting as the root, s2 and s3 as the aggregation layer, and the remaining switches as the edge layer connecting to the hosts.
- The network is built, the remote controller is started, and each of the seven switches is sequentially connected to the controller before handing execution over to the Mininet CLI.

2.1.2 Results and Command Execution

To deploy this topology and integrate it with OpenDaylight, the following commands were executed sequentially across Linux terminals:

1. `./bin/karaf`: Executed within the OpenDaylight directory to launch the controller's interactive console.
2. `feature:install odl-dluxapps-applications odl-restconf-all odl-l2switch-all odl-openflowplugin`: Run inside the Karaf console to install the necessary RESTCONF APIs, DLUX web GUI, and the automated L2 learning switch logic.
3. `sudo python3 create_tree_net.py`: Executed in a separate terminal to build the Mininet topology and connect the Open vSwitches to ODL.
4. `pingall`: Executed within the Mininet CLI to verify that OpenDaylight's l2switch module successfully mapped the network and established full connectivity.
5. `sh sudo ovs-ofctl -O OpenFlow13 dump-flows s1`: Executed within the Mininet CLI to extract and inspect the flow tables automatically generated by OpenDaylight for the root switch.

```

at org.eclipse.osgi.internal.framework.BundleContextImpl.start(BundleContextImpl.java:721)
at org.eclipse.osgi.internal.framework.EquinoxBundle.startWorker0(EquinoxBundle.java:941)
at org.eclipse.osgi.internal.framework.EquinoxBundle$EquinoxModule.startWorker(EquinoxBundle.java:318)
at org.eclipse.osgi.container.Module.doStart(Module.java:571)
at org.eclipse.osgi.container.Module.start(Module.java:439)
at org.eclipse.osgi.container.ModuleContainer$ContainerStartLevel.incStartLevel(ModuleContainer.java:1582)
at org.eclipse.osgi.container.ModuleContainer$ContainerStartLevel.incStartLevel(ModuleContainer.java:1562)
at org.eclipse.osgi.container.ModuleContainer$ContainerStartLevel.doContainerStartLevel(ModuleContainer.java:153
3)
at org.eclipse.osgi.container.ModuleContainer$ContainerStartLevel.dispatchEvent(ModuleContainer.java:1476)
at org.eclipse.osgi.container.ModuleContainer$ContainerStartLevel.dispatchEvent(ModuleContainer.java:1)
at org.eclipse.osgi.framework.eventmgr.EventManager.dispatchEvent(EventManager.java:230)
at org.eclipse.osgi.framework.eventmgr.EventManager$EventThread.run(EventManager.java:340)

100% [=====]
Karaf started in 1s. Bundle stats: 53 active, 54 total

Hit '<tab>' for a list of available commands
and '[cmd] --help' for help on a specific command.
Hit '<ctrl-d>' or type 'system:shutdown' or 'logout' to shutdown OpenDaylight.

opendaylight-user@root>

```

Figure 7: OpenDaylight Nitrogen controller initialized in the Karaf console.

The terminal output demonstrates the successful startup of the OpenDaylight Nitrogen controller via the Karaf container. The log indicates that the required OSGi bundles and features, including the OpenFlow plugins and L2 switch modules, have been actively loaded into the environment without any fatal exceptions, confirming that the controller is ready to accept connections from Mininet.

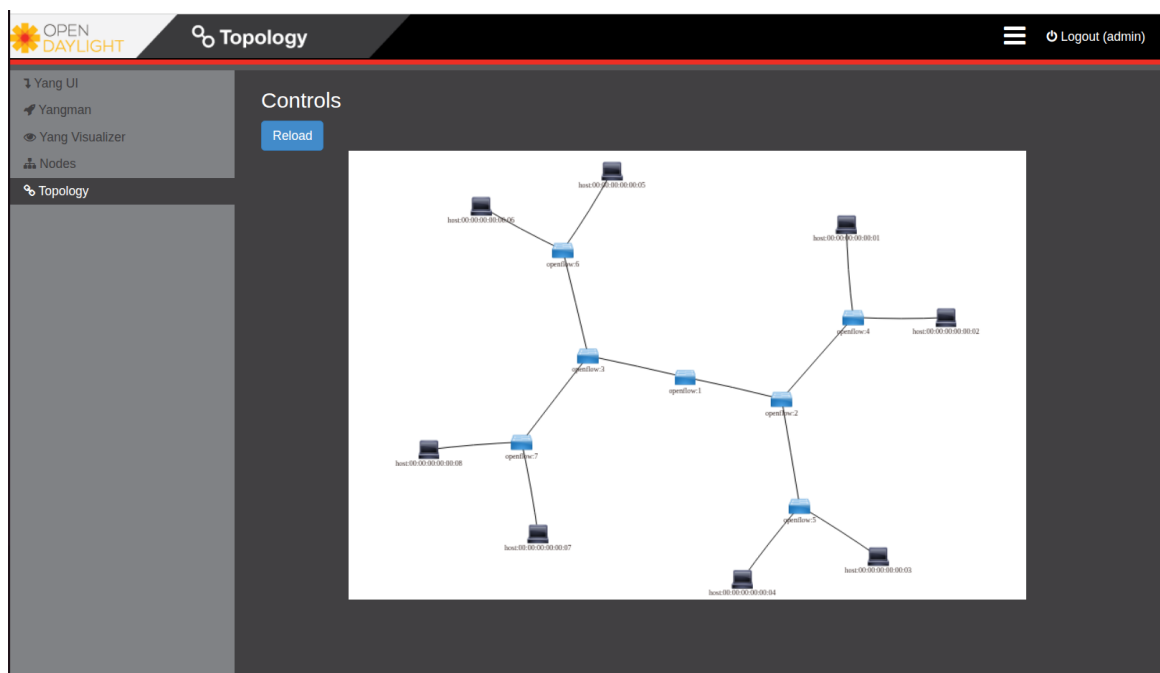


Figure 8: Topology discovery visualized via the OpenDaylight DLUX GUI.

This screenshot displays the DLUX web interface provided by ODL, specifically the Topology viewer. The controller successfully utilized the Link Layer Discovery Protocol (LLDP) to map the entire network. The visual graph accurately reflects our Python script, showing the root switch (openflow:1) connected to two aggregation switches, branching out to the edge switches and their respective hosts, justifying that ODL is fully aware of the 3-level tree structure.

```

inan@inan-Virtual-Platform:~$ sudo python3 create_tree_net.py
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>

```

Figure 9: Mininet CLI executing a global ping test across the tree topology.

The pingall command execution confirms that all 8 hosts can successfully communicate with one another across the 7 switches. The result shows 56/56 connections received with a 0% drop rate. This proves that the od1-12switch-all feature in OpenDaylight successfully functioned as a reactive controller, listening to packet-ins, learning MAC addresses, and dynamically pushing the correct forwarding rules to all switches in the tree.

```

mininet> sh sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x2b00000000000007, duration=1564.075s, table=0, n_packets=627, n_bytes=53295, priority=100,dl_type=0x88cc actions=CONTROLLER:65535
cookie=0x2b000000000000026, duration=1558.062s, table=0, n_packets=211, n_bytes=19576, priority=2,in_port="s1-eth2" actions=output:"s1-eth1"
cookie=0x2b000000000000027, duration=1558.061s, table=0, n_packets=210, n_bytes=19586, priority=2,in_port="s1-eth1" actions=output:"s1-eth2"
cookie=0x2b000000000000007, duration=1564.075s, table=0, n_packets=5, n_bytes=613, priority=0 actions=drop

```

Figure 10: OpenFlow rules dynamically injected by OpenDaylight into the root switch (s1).

This output reveals the actual flow table of the root switch (s1) generated by ODL. The highest priority flow (priority=100) matches on dl_type=0x88cc (LLDP packets) and sends them directly to the CONTROLLER, which allows ODL to continuously monitor link states. The lower priority flows (priority=2) are the dynamically learned L2 forwarding rules, matching traffic arriving on in_port="s1-eth2" and outputting it to s1-eth1, and vice versa. This confirms that ODL successfully translated its global network view into functional OpenFlow instructions on the physical hardware.

2.2 Part 2: Using RESTCONF

In this section, we migrate the subnet-routing architecture from Question 1, Part 2 under the complete control of the OpenDaylight SDN controller. To achieve routing across different subnets, the internal automated 12switch application is disabled to prevent rule conflicts. Instead, we program an external routing controller application using OpenDaylight's Northbound RESTCONF API. This external script constructs JSON payloads describing explicit Layer 2 and Layer 3 flow entries and pushes them to the switches via PUT requests targeting the controller's operational configuration datastore.

2.2.1 Codes and Implementation

```

1  from mininet.net import Mininet
2  from mininet.node import RemoteController, OVSKernelSwitch
3  from mininet.cli import CLI
4  from mininet.log import setLogLevel
5  from mininet.link import TCLink
6
7  def topology():
8      net = Mininet(controller=RemoteController, switch=OVSKernelSwitch,
9                  link=TCLink)
10
11     c0 = net.addController('c0', controller=RemoteController,
12                           ip='127.0.0.1', port=6633)
13     h1 = net.addHost('h1', ip='10.0.1.1/24', defaultRoute='via
14                     10.0.1.254', mac='00:00:00:00:01:01')

```

```

12 h2 = net.addHost('h2', ip='10.0.2.1/24', defaultRoute='via
    10.0.2.254', mac='00:00:00:00:02:01')
13 s1 = net.addSwitch('s1', protocols="OpenFlow13",
    dpid="0000000000000001") # openflow:1
14 s2 = net.addSwitch('s2', protocols="OpenFlow13",
    dpid="0000000000000002") # openflow:2
15 r1 = net.addSwitch('r1', protocols="OpenFlow13",
    dpid="0000000000000003") # openflow:3
16
17 net.addLink(h1, s1, port1=0, port2=1)
18 net.addLink(s1, r1, port1=2, port2=1)
19 net.addLink(r1, s2, port1=2, port2=1)
20 net.addLink(s2, h2, port1=2, port2=0)
21
22 net.build()
23 c0.start()
24 s1.start([c0]); s2.start([c0]); r1.start([c0])
25
26 h1.cmd('arp -s 10.0.1.254 00:00:00:00:01:fe')
27 h2.cmd('arp -s 10.0.2.254 00:00:00:00:02:fe')
28
29 CLI(net)
30 net.stop()
31
32 if __name__ == '__main__':
33     setLogLevel('info')
34     topology()

```

Listing 6: create_odl_router.py: Network Definition under ODL Control

- The script instantiates a Mininet network attached to a RemoteController representing our OpenDaylight instance operating on port 6633.
- Two hosts are established in separate subnets (10.0.1.1/24 and 10.0.2.1/24), complete with explicit hardcoded hardware addresses and default gateway routes.
- Three switches are generated with forced Datapath IDs (dpid) matching openflow:1, openflow:2, and openflow:3 respectively to permit deterministic tracking across RESTCONF endpoints.
- Inter-switch links and host connections are established following a linear multi-hop scheme, and all nodes are initialized under the control of the c0 loopback controller.
- Static gateway ARP bindings are forced directly into the host namespaces via backend shell execution to bypass native subnet broadcast limitations.

```

1 import json
2 import os
3
4 if not os.path.exists('flows'):
5     os.makedirs('flows')
6
7 def write_flow(node, flow_id, data):
8     filename = f"flows/node_{node}_flow_{flow_id}.json"
9     with open(filename, 'w') as f:
10         json.dump(data, f, indent=4)
11     print(f"Created: {filename}")
12

```

```

13 def gen_l2_flow(flow_id, in_port, out_port):
14     return {
15         "flow": [{
16             "id": str(flow_id), "table_id": 0, "priority": 100,
17             "match": { "in-port": str(in_port) },
18             "instructions": { "instruction": [{ "order": 0,
19                 "apply-actions": { "action": [{ "order": 0,
20                     "output-action": { "output-node-connector":
21                         str(out_port) } }
22                 } ] }
23         } ]
24     }
25
26 def gen_l3_flow(flow_id, dst_ip, src_mac, dst_mac, out_port):
27     return {
28         "flow": [{
29             "id": str(flow_id), "table_id": 0, "priority": 100,
30             "match": {
31                 "ethernet-match": { "ethernet-type": { "type": 2048 } },
32                 "ipv4-destination": dst_ip
33             },
34             "instructions": { "instruction": [{ "order": 0,
35                 "apply-actions": { "action": [
36                     { "order": 0, "set-dl-src-action": { "address":
37                         src_mac } },
38                     { "order": 1, "set-dl-dst-action": { "address":
39                         dst_mac } },
40                     { "order": 2, "dec-nw-ttl": {} },
41                     { "order": 3, "output-action": {
42                         "output-node-connector": str(out_port) } }
43                 ] }
44             } ] }
45         } ]
46     }
47
48 print("--- Generating ODL RESTCONF Payloads ---")
49
50 write_flow("1", "1", gen_l2_flow("1", "1", "2"))
51 write_flow("1", "2", gen_l2_flow("2", "2", "1"))
52
53 write_flow("2", "1", gen_l2_flow("1", "1", "2"))
54 write_flow("2", "2", gen_l2_flow("2", "2", "1"))
55
56 write_flow("3", "1", gen_l3_flow("1", "10.0.2.1/32", "00:00:00:00:02:fe",
57     "00:00:00:00:02:01", "2"))
58 write_flow("3", "2", gen_l3_flow("2", "10.0.1.1/32", "00:00:00:00:01:fe",
59     "00:00:00:00:01:01", "1"))
60
61 print("All flows successfully saved to the 'flows/' folder.")

```

Listing 7: create_flows.py: JSON Payload Generation Script

- An automated runtime verification check creates a localized directory named flows if it is missing from the working directory.
- The gen_l2_flow helper generates standardized OpenFlow 1.3 entries mapped to localized switch port identifiers for strict ingress/egress physical tracking.
- The gen_l3_flow engine acts as our structural routing model, matching incoming IPv4 headers

(type=2048) and appending specialized, sequential modifications.

- Actions for Layer 3 flows manipulate data link characteristics by rewriting source and destination MAC boundaries, decrementing packet TTL metrics, and sending traffic forward.
- Six structural components are systematically exported to localized JSON configurations representing the complete bidirectional rule layout across our switches and virtual routing hop.

```
1 import os
2 import json
3 import requests
4
5 ODL_IP = "127.0.0.1"
6 ODL_PORT = "8181"
7 AUTH = ('admin', 'admin')
8 HEADERS = {'Content-Type': 'application/json', 'Accept':
9            'application/json'}
10
11 flow_dir = 'flows'
12
13 print("--- Pushing Flows to OpenDaylight ---")
14
15 for filename in os.listdir(flow_dir):
16     if filename.endswith(".json"):
17         filepath = os.path.join(flow_dir, filename)
18         parts = filename.split('_')
19         node_id = parts[1]
20         flow_id = parts[3].split('.')[0]
21
22         url =
23             f"http://{ODL_IP}:{ODL_PORT}/restconf/config/.opendaylight-inventory:nodes
24
25         with open(filepath, 'r') as f:
26             flow_data = json.load(f)
27             response = requests.put(url, auth=AUTH, headers=HEADERS,
28                                    json=flow_data)
29
30         if response.status_code in [200, 201]:
31             print(f"SUCCESS: Pushed {filename} to openflow:{node_id}")
32         else:
33             print(f"FAILED: Could not push {filename}. Status Code:
34                   {response.status_code}")
35             print(response.text)
```

Listing 8: push_flows.py: Northbound RESTCONF Driver Script

- This automation script parses the target storage folder to discover valid payload structures dynamically.
- String tokenization breaks down file structures to capture contextual properties including the target node_id and corresponding unique OpenFlow flow_id entries.
- Formatted string templates compile target URLs pointing to the exact target inventory config endpoints of the active MD-SAL architecture inside OpenDaylight.
- HTTP PUT operations carry raw file inputs to inject custom parameters into individual flow tables securely under default administrator credentials.

- The script inspects server execution tokens, evaluating successful entries (200 or 201 OK codes) to report status validation messages back to the primary console window.

2.2.2 Flow Configurations and Analysis

Due to file transfer constraints, the full list of generated configurations inside the `flows/` subdirectory is presented textually below. These JSON payloads establish deterministic, bidirectional flow matching using an OpenFlow 1.3 pipeline format across all switches.

Edge Switch 1 rules (`node_1_flow_1.json` and `node_1_flow_2.json`) These entries map simple physical layer requirements. Inward traffic arriving on the host-facing port (`in-port=1`) is cleanly transmitted to the router on port 2. Conversely, return frames coming from the router interface (`in-port=2`) are mapped directly to port 1 to ensure safe transit back to host h1.

Edge Switch 2 rules (`node_2_flow_1.json` and `node_2_flow_2.json`) Similar to switch 1, these instructions govern the physical boundaries for host h2. Traffic entering via port 1 is forwarded to the router links on port 2, while return traffic from the router (`in-port=2`) is output back to host h2 on port 1.

Central Router rules (`node_3_flow_1.json` and `node_3_flow_2.json`) The router node (`openflow:3`) handles the Layer 3 transformations. For forward traffic aimed at host h2 (`10.0.2.1/32`), the router changes the source MAC address to its own outgoing interface MAC (`00:00:00:00:02:fe`) and updates the destination MAC address to match host h2's hardware address (`00:00:00:00:02:01`). It then decrements the IP Time-To-Live (`dec-nw-ttl`) and forwards the frame out port 2.

For reverse path traffic returning to host h1 (`10.0.1.1/32`), the router executes the inverse operation:

```

1  {
2      "flow": [
3          {
4              "id": "2",
5              "table_id": 0,
6              "priority": 100,
7              "match": {
8                  "ethernet-match": {
9                      "ethernet-type": {
10                         "type": 2048
11                     }
12                 },
13                 "ipv4-destination": "10.0.1.1/32"
14             },
15             "instructions": {
16                 "instruction": [
17                     {
18                         "order": 0,
19                         "apply-actions": {
20                             "action": [
21                                 {
22                                     "order": 0,
23                                     "set-dl-src-action": {
24                                         "address": "00:00:00:00:01:fe"
25                                     }
26                                 },
27                                 {
28                                     "order": 1,
29                                     "set-dl-dst-action": {
30                                         "address": "00:00:00:00:01:01"
31                                     }
32                                 }
21         ]
22         }
23         }
24         ]
25     }
26     ]
27     }
28     ]
29     ]
30     ]

```

```

31         }
32     },
33     {
34         "order": 2,
35         "dec-nw-ttl": {}
36     },
37     {
38         "order": 3,
39         "output-action": {
40             "output-node-connector": "1"
41         }
42     }
43 ]
44 }
45 }
46 ]
47 }
48 }
49 ]
50 }

```

Listing 9: node_3_flow_2.json: Reverse Path Routing Payload

This file establishes a precise match for IPv4 destination traffic returning to 10.0.1.1/32. The switch updates the source hardware layer address to the target gateway boundary (00:00:00:00:01:fe), updates the destination link address to target host h1 directly (00:00:00:00:01:01), decrements the packet's network TTL to indicate a physical routing hop, and routes the packet out interface port 1.

2.2.3 Results and Command Execution

To compile our custom flow rules and inject them into the active topology, the following sequence of commands was executed across separate Linux shell panels:

1. `python3 create_flows.py`: Executes the payload generator script to output the structured JSON rule models into the localized file store.
2. `python3 push_flows.py`: Launches the Northbound RESTCONF controller program to transmit our network rules directly into the OpenDaylight configuration database.
3. `sudo python3 create_odl_router.py`: Launches the specialized Mininet orchestration infrastructure, joining the isolated switches to our operational remote SDN controller.
4. `pingall`: Executed within the Mininet CLI instance to confirm network reachability across all subnet components.

```

inan@inan-Virtual-Platform:~$ python3 create_flows.py
--- Generating ODL RESTCONF Payloads ---
Created: flows/node_1_flow_1.json
Created: flows/node_1_flow_2.json
Created: flows/node_2_flow_1.json
Created: flows/node_2_flow_2.json
Created: flows/node_3_flow_1.json
Created: flows/node_3_flow_2.json
All flows successfully saved to the 'flows/' folder.
inan@inan-Virtual-Platform:~$ python3 push_flows.py
--- Pushing Flows to OpenDaylight ---
SUCCESS: Pushed node_2_flow_2.json to openflow:2
SUCCESS: Pushed node_2_flow_1.json to openflow:2
SUCCESS: Pushed node_3_flow_1.json to openflow:3
SUCCESS: Pushed node_1_flow_2.json to openflow:1
SUCCESS: Pushed node_1_flow_1.json to openflow:1
SUCCESS: Pushed node_3_flow_2.json to openflow:3
inan@inan-Virtual-Platform:~$

```

Figure 11: JSON payload generation and execution of the RESTCONF injection script.

The console logs show the successful creation and deployment of our custom flow rules. Executing `create_flows.py` yields six sequential JSON files representing the network paths. Following this, the

driver execution script (`push_flows.py`) correctly processes the parameters, generating unambiguous server responses marked with structural SUCCESS confirmation outputs for every targeted openflow node container (`openflow:1`, `openflow:2`, and `openflow:3`).

```
inan@inan-Virtual-Platform:~$ sudo python3 create_odl_router.py
*** Configuring hosts
h1 h2
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Figure 12: Mininet verification showing successful network connectivity.

The global diagnostic test execution validates the end-to-end operation of our configuration. Running `pingall` within the Mininet environment reports a successful link test between host `h1` and host `h2`, resulting in a perfect 0% dropped (2/2 received) metric. This confirms that the custom flow rules pushed via the RESTCONF API are successfully managing the network data plane, routing inter-subnet traffic through the central router node without the assistance of any automated localized discovery applications.

3 Problem 3: Minimum Weight Routing

In this problem, we implement a dynamic minimum cost path routing algorithm to connect two hosts located in different subnets across an asymmetrical network. Using a provided adjacency matrix A , we compute the shortest path utilizing Dijkstra's algorithm, automatically translate these paths into OpenFlow Layer 3 and Layer 2 instructions, and inject them into the OpenDaylight controller via its RESTCONF API.

3.1 Codes and Implementation

```
1  import networkx as nx
2
3  A = [
4      [0, 2, 3, 4],
5      [2, 0, 0, 1],
6      [3, 0, 0, 0],
7      [1, 1, 0, 0]
8  ]
9
10 def get_paths(matrix):
11     n = len(matrix)
12     G = nx.DiGraph()
13
14     for i in range(n):
15         for j in range(n):
16             if matrix[i][j] > 0:
17                 G.add_edge(i+1, j+1, weight=matrix[i][j])
18
19     path_fwd = nx.shortest_path(G, source=1, target=n, weight='weight')
20     path_rev = nx.shortest_path(G, source=n, target=1, weight='weight')
21
22     return path_fwd, path_rev
23
24
25 if __name__ == '__main__':
26     p1, p2 = get_paths(A)
27     print("Path 1 -> n:", " -> ".join([f"s{x}" for x in p1]))
28     print("Path n -> 1:", " -> ".join([f"s{x}" for x in p2]))
```

Listing 10: spf.py: Shortest Path Calculation

- This script defines the static adjacency matrix A , representing the directed link weights between the network switches.
- It utilizes the networkx library to construct a directed graph (`nx.DiGraph`) and populates it with edges only where the matrix weight is greater than zero.
- The `nx.shortest_path` function applies Dijkstra's algorithm to calculate the optimal forward route from switch 1 to switch n , and the reverse route from switch n to switch 1 based on the edge weights.

```
1  import json
2  import os
3  from spf import get_paths, A
4
```

```

5  n = len(A)
6  path_fwd, path_rev = get_paths(A)
7
8  if not os.path.exists('flows'):
9      os.makedirs('flows')
10
11 def write_flow(node, flow_id, data):
12     filename = f"flows/node_{node}_flow_{flow_id}.json"
13     with open(filename, 'w') as f:
14         json.dump(data, f, indent=4)
15
16 def gen_l3_flow(flow_id, dst_ip, src_mac, dst_mac, out_port):
17     return {
18         "flow": [{
19             "id": str(flow_id), "table_id": 0, "priority": 100,
20             "match": {
21                 "ethernet-match": { "ethernet-type": { "type": 2048 } },
22                 "ipv4-destination": dst_ip
23             },
24             "instructions": { "instruction": [{ "order": 0,
25                 "apply-actions": { "action": [
26                     { "order": 0, "set-dl-src-action": { "address":
27                         src_mac } },
28                     { "order": 1, "set-dl-dst-action": { "address":
29                         dst_mac } },
30                     { "order": 2, "dec-nw-ttl": {} },
31                     { "order": 3, "output-action": {
32                         "output-node-connector": str(out_port) } }
33                 ]}
34             }
35         ]}
36     ]}
37
38 def gen_fwd_flow(flow_id, dst_ip, out_port):
39     return {
40         "flow": [{
41             "id": str(flow_id), "table_id": 0, "priority": 50,
42             "match": {
43                 "ethernet-match": { "ethernet-type": { "type": 2048 } },
44                 "ipv4-destination": dst_ip
45             },
46             "instructions": { "instruction": [{ "order": 0,
47                 "apply-actions": { "action": [
48                     { "order": 0, "output-action": {
49                         "output-node-connector": str(out_port) } }
50                 ]}
51             }
52         ]}
53     ]}
54
55 print("--- Generating ODL RESTCONF Payloads based on Shortest Path ---")
56 flow_counters = {i: 1 for i in range(1, n+1)}
57
58 dst_ip_fwd = "10.0.2.1/32"
59 for idx, node in enumerate(path_fwd):
60     if idx == 0:
61         out_p = path_fwd[idx+1]
62         flow = gen_l3_flow(flow_counters[node], dst_ip_fwd,
63             "00:00:00:00:02:fe", "00:00:00:00:02:01", out_p)
64     elif idx == len(path_fwd) - 1:
65         out_p = n + 1

```

```

61         flow = gen_fwd_flow(flow_counters[node], dst_ip_fwd, out_p)
62     else:
63         out_p = path_fwd[idx+1]
64         flow = gen_fwd_flow(flow_counters[node], dst_ip_fwd, out_p)
65
66     write_flow(node, flow_counters[node], flow)
67     flow_counters[node] += 1
68
69 dst_ip_rev = "10.0.1.1/32"
70 for idx, node in enumerate(path_rev):
71     if idx == 0:
72         out_p = path_rev[idx+1]
73         flow = gen_l3_flow(flow_counters[node], dst_ip_rev,
74                             "00:00:00:00:01:fe", "00:00:00:00:01:01", out_p)
75     elif idx == len(path_rev) - 1:
76         out_p = n + 1
77         flow = gen_fwd_flow(flow_counters[node], dst_ip_rev, out_p)
78     else:
79         out_p = path_rev[idx+1]
80         flow = gen_fwd_flow(flow_counters[node], dst_ip_rev, out_p)
81
82     write_flow(node, flow_counters[node], flow)
83     flow_counters[node] += 1
84
85 print(f"All flows generated in 'flows/' directory.")

```

Listing 11: create_flows.py: Dynamic Flow Generation

- This script acts as the flow compiler, importing the calculated shortest paths from `spf.py`.
- It defines two distinct flow generation functions: `gen_l3_flow` creates high-priority Layer 3 routing instructions that rewrite MAC addresses and decrement the TTL, while `gen_fwd_flow` creates lower-priority Layer 2 instructions that simply match the IP and forward the packet.
- The script iterates through the forward and reverse path arrays, strategically assigning the complex L3 flow to the first (ingress) node in the path, and assigning the simpler L2 forwarding flows to all intermediate and egress nodes.
- Every generated configuration is cleanly serialized into JSON format and saved in a localized `flows/` directory with an identifiable naming convention.

```

1  import os
2  import json
3  import requests
4
5  ODL_IP = "127.0.0.1"
6  ODL_PORT = "8181"
7  AUTH = ('admin', 'admin')
8  HEADERS = {'Content-Type': 'application/json', 'Accept':
9             'application/json'}
10 flow_dir = 'flows'
11
12 print("--- Sending Minimum Weight Flows to OpenDaylight ---")
13
14 for filename in os.listdir(flow_dir):
15     if filename.endswith(".json"):
16         filepath = os.path.join(flow_dir, filename)

```

```

16     parts = filename.split('_')
17     node_id = parts[1]
18     flow_id = parts[3].split('.')[0]
19
20
21     url =
22         f"http://{ODL_IP}:{ODL_PORT}/restconf/config/opendaylight-inventory:nodes
23
24     with open(filepath, 'r') as f:
25         flow_data = json.load(f)
26
27     response = requests.put(url, auth=AUTH, headers=HEADERS,
28                             json=flow_data)
29
30     if response.status_code in [200, 201]:
31         print(f"SUCCESS: Pushed {filename} to openflow:{node_id}")
32     else:
33         print(f"FAILED: {filename}. Status Code:
34             {response.status_code}")

```

Listing 12: send_flows.py: RESTCONF Flow Injection

- This script parses the flows/ directory, opening each JSON file sequentially.
- It extracts the target node_id and flow_id directly from the filename to construct the precise OpenDaylight RESTCONF endpoint URL.
- A PUT request containing the JSON payload is transmitted to the controller, and the script logs the HTTP status codes to confirm successful injection into the operational datastore.

```

1  from mininet.net import Mininet
2  from mininet.node import RemoteController, OVSKernelSwitch
3  from mininet.cli import CLI
4  from mininet.log import setLogLevel
5  from mininet.link import TCLink
6  from spf import A
7
8  def topology():
9      n = len(A)
10     net = Mininet(controller=RemoteController, switch=OVSKernelSwitch,
11                  link=TCLink)
12     c0 = net.addController('c0', controller=RemoteController,
13                           ip='127.0.0.1', port=6633)
14     switches = {}
15
16     for i in range(1, n+1):
17         switches[i] = net.addSwitch(f's{i}', protocols="OpenFlow13",
18                                   dpid=str(i).zfill(16))
19
20     h1 = net.addHost('h1', ip='10.0.1.1/24', defaultRoute='via
21                    10.0.1.254', mac='00:00:00:00:01:01')
22     h2 = net.addHost('h2', ip='10.0.2.1/24', defaultRoute='via
23                    10.0.2.254', mac='00:00:00:00:02:01')
24
25     for i in range(1, n+1):
26         for j in range(i+1, n+1):
27             if A[i-1][j-1] > 0 or A[j-1][i-1] > 0:

```

```

23         net.addLink(switches[i], switches[j], port1=j, port2=i)
24
25     net.addLink(h1, switches[1], port1=0, port2=n+1)
26     net.addLink(h2, switches[n], port1=0, port2=n+1)
27
28     net.build()
29     c0.start()
30
31     for i in range(1, n+1):
32         switches[i].start([c0])
33
34     h1.cmd('arp -s 10.0.1.254 00:00:00:00:01:fe')
35     h2.cmd('arp -s 10.0.2.254 00:00:00:00:02:fe')
36
37     CLI(net)
38     net.stop()
39
40 if __name__ == '__main__':
41     setLogLevel('info')
42     topology()

```

Listing 13: create_net.py: Topology Generation from Matrix

- This script dynamically constructs the Mininet topology by importing the adjacency matrix A directly from the `spf.py` module.
- It calculates the total number of switches n , generates them with deterministic datapath IDs, and iterates through the matrix to establish links wherever a weight greater than zero exists.
- Two hosts are provisioned in entirely different subnets with explicit default routes, attached to the edge switches s_1 and s_n , and configured with static ARP tables to eliminate broadcast resolution issues.

3.2 Flow Configurations and Analysis

Our programmatic approach generates two distinct categories of OpenFlow rules to handle cross-subnet communication efficiently across the calculated path:

Layer 3 Ingress Flows: Flows such as `node3-flow1`, `node3-flow2`, and the generated payload matching IPv4 destination `10.0.2.1/32` serve as the primary routing mechanism. These high-priority rules (priority 100) are placed on the ingress switches. When a packet arrives, the flow rewrites the Data Link Source Address (`set-dl-src-action`) to emulate the gateway MAC, rewrites the Data Link Destination Address (`set-dl-dst-action`) to target the receiving host, and strictly executes a `dec-nw-ttl` instruction to mimic a physical router hop.

Layer 2 Forwarding Flows: Flows distributed to the intermediate and egress switches feature a lower priority (priority 50). Because the ingress switch already handled the complex Layer 3 MAC translations and TTL decrements, these intermediate nodes merely need to match the destination IP address and perform a simple `output-action` pointing to the precise node connector calculated by the shortest path algorithm.

3.3 Results and Command Execution

To deploy the minimum-weight routing infrastructure, the following terminal commands were executed in sequence:

1. `sudo python3 spf.py`: Calculates and prints the optimal forward and reverse routes.
2. `sudo python3 create_flows.py`: Parses the calculated paths to generate the JSON configurations.
3. `sudo python3 send_flows.py`: Pushes the JSON flow definitions to the active OpenDaylight instance via RESTCONF.
4. `sudo python3 create_net.py`: Deploys the Mininet virtual topology connected to the pre-configured controller.
5. `pingall`: Verifies global reachability across the matrix topology.

```
lman@lman-Virtual-Platform:~$ sudo python3 spf.py
[sudo] password for lman:
Path 1 -> n: s1 -> s2 -> s4
Path n -> 1: s4 -> s1
lman@lman-Virtual-Platform:~$ sudo python3 create_flows.py
sudo: python: command not found
lman@lman-Virtual-Platform:~$ sudo python3 create_flows.py
--- Generating ODL RESTCONF Payloads based on Shortest Path ---
All flows generated in 'flows/' directory.
lman@lman-Virtual-Platform:~$ sudo python3 send_flows.py
--- Sending Minimum Weight Flows to OpenDaylight ---
SUCCESS: Pushed node_2_flow_1.json to openflow:2
SUCCESS: Pushed node_4_flow_2.json to openflow:4
SUCCESS: Pushed node_1_flow_2.json to openflow:1
SUCCESS: Pushed node_1_flow_1.json to openflow:1
SUCCESS: Pushed node_4_flow_1.json to openflow:4
lman@lman-Virtual-Platform:~$
```

Figure 13: Shortest path calculation, flow generation, and RESTCONF transmission.

The terminal output captures the entire automated pipeline. Initially, the `spf.py` execution reveals the calculated minimum weight paths: `s1 -> s2 -> s4` for the forward route, and `s4 -> s1` for the reverse route. Subsequently, the `create_flows.py` script confirms the successful serialization of these paths into JSON payloads. Finally, the `send_flows.py` script executes, returning `SUCCESS` status messages for every individual file, verifying that OpenDaylight accepted the custom routing structure.

```
lman@lman-Virtual-Platform:~$ sudo python3 create_net.py
*** Configuring hosts
h1 h2
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Figure 14: Mininet validation of the minimum weight routing topology.

Executing the `pingall` command within Mininet demonstrates full operational connectivity. The test yields 0% dropped (2/2 received), proving definitively that hosts `h1` and `h2` can communicate across their isolated subnets. The custom OpenFlow instructions successfully navigated the packets along the lowest-cost paths computed by Dijkstra's algorithm, completing the requirements for dynamic, minimum-weight SDN routing.

4 Problem 4: Dynamic Routing (Bonus)

In this final section, we extend the minimum weight routing architecture to support dynamic fault tolerance. The network is monitored continuously by a custom Python daemon that queries the OpenDaylight RESTCONF inventory. If a physical link failure is detected, the system automatically purges the stale flow entries, recalculates the shortest path across the surviving links using Dijkstra's algorithm, and injects the new recovery flows into the switches.

4.1 Codes and Implementation

```
1  from mininet.net import Mininet
2  from mininet.node import RemoteController, OVSKernelSwitch
3  from mininet.cli import CLI
4  from mininet.log import setLogLevel
5  import time
6
7  A = [
8      [0, 2, 3, 4],
9      [2, 0, 0, 1],
10     [3, 0, 0, 0],
11     [1, 1, 0, 0]
12 ]
13
14 def topology():
15     net = Mininet(controller=RemoteController, switch=OVSKernelSwitch)
16     c1 = net.addController('c1', ip='127.0.0.1', port=6633)
17
18     n = len(A)
19     switches = []
20
21     for i in range(n):
22         dpid = "%016x" % (i+1)
23         switches.append(net.addSwitch('s%d' % (i+1),
24                                     protocols='OpenFlow13', dpid=dpid))
25
26     h1 = net.addHost('h1', ip='10.0.1.10/24', mac='00:00:00:00:00:01',
27                     defaultRoute='via 10.0.1.1')
28     hn = net.addHost('h%d' % n, ip='10.0.2.10/24',
29                     mac='00:00:00:00:00:02', defaultRoute='via 10.0.2.1')
30
31     net.addLink(h1, switches[0])
32     net.addLink(hn, switches[-1])
33
34     for i in range(n):
35         for j in range(i+1, n):
36             if A[i][j] > 0 or A[j][i] > 0:
37                 net.addLink(switches[i], switches[j])
38
39     net.build()
40     net.start()
41
42     time.sleep(2)
43
44     h1.cmd('arp -s 10.0.1.1 00:00:00:00:00:02')
45     hn.cmd('arp -s 10.0.2.1 00:00:00:00:00:01')
46
47     CLI(net)
48     net.stop()
```

```

47 if __name__ == '__main__':
48     setLogLevel('info')
49     topology()

```

Listing 14: create_net.py: Matrix-Driven Mininet Topology

- This script initializes a Mininet topology driven by an adjacency matrix, ensuring switches are created with explicit 16-character Datapath IDs for precise tracking.
- Hosts are intentionally placed in completely separate IP subnets with default gateway routes.
- Static ARP entries are injected directly into the hosts, bypassing standard ARP broadcast limitations to facilitate seamless Layer 3 routing.

```

1  import networkx as nx
2
3  def calculate_spf(A, active_links, n):
4      G = nx.DiGraph()
5      G.add_nodes_from(range(1, n + 1))
6
7      for i in range(n):
8          for j in range(n):
9              if A[i][j] > 0:
10                 if (i+1, j+1) in active_links:
11                     G.add_edge(i+1, j+1, weight=A[i][j])
12
13     try:
14         path_1_n = nx.shortest_path(G, source=1, target=n,
15                                     weight='weight')
16     except nx.NetworkXNoPath:
17         path_1_n = []
18
19     try:
20         path_n_1 = nx.shortest_path(G, source=n, target=1,
21                                     weight='weight')
22     except nx.NetworkXNoPath:
23         path_n_1 = []
24
25     return path_1_n, path_n_1

```

Listing 15: spf.py: Dynamic Shortest Path Calculation

- The shortest path function accepts the base adjacency matrix alongside a dynamically updated list of active_links.
- It uses the networkx library to build a directed graph, strictly ignoring any edges that correspond to dead or severed links.
- The algorithm safely catches nx.NetworkXNoPath exceptions, returning empty arrays if the network becomes completely partitioned.

```

1  import json
2  import os

```

```

3
4 def create_flow_json(flow_id, dest_ip, out_port):
5     flow = {
6         "flow": [
7             {
8                 "id": str(flow_id),
9                 "table_id": "0",
10                "priority": "100",
11                "match": {
12                    "ethernet-match": {
13                        "ethernet-type": { "type": 2048 }
14                    },
15                    "ipv4-destination": dest_ip
16                },
17                "instructions": {
18                    "instruction": [
19                        {
20                            "order": 0,
21                            "apply-actions": {
22                                "action": [
23                                    {
24                                        "order": 0,
25                                        "dec-nw-ttl": {}
26                                    },
27                                    {
28                                        "order": 1,
29                                        "output-action": {
30                                            "output-node-connector":
31                                                str(out_port)
32                                        }
33                                    }
34                                ]
35                            }
36                        }
37                    ]
38                }
39            }
40        ]
41        return flow
42
43 def save_flows(path_1_n, path_n_1, port_mapping):
44     if not os.path.exists("flows"):
45         os.makedirs("flows")
46
47     flow_files = []
48
49     for i in range(len(path_1_n)):
50         current_node = path_1_n[i]
51         if i == len(path_1_n) - 1:
52             out_port = 1
53         else:
54             next_node = path_1_n[i+1]
55             out_port = port_mapping.get((current_node, next_node))
56
57         flow_data = create_flow_json(1, "10.0.2.10/32", out_port)
58         filename = "flows/node_%s_forward.json" % current_node
59         with open(filename, 'w') as f:
60             json.dump(flow_data, f, indent=4)
61         flow_files.append((current_node, 1, filename))
62

```

```

63     for i in range(len(path_n_1)):
64         current_node = path_n_1[i]
65         if i == len(path_n_1) - 1:
66             out_port = 1
67         else:
68             next_node = path_n_1[i+1]
69             out_port = port_mapping.get((current_node, next_node))
70
71         flow_data = create_flow_json(2, "10.0.1.10/32", out_port)
72         filename = "flows/node_%s_reverse.json" % current_node
73         with open(filename, 'w') as f:
74             json.dump(flow_data, f, indent=4)
75         flow_files.append((current_node, 2, filename))
76
77     return flow_files

```

Listing 16: create_flows.py: JSON Flow Payload Generator

- This script dynamically constructs OpenFlow 1.3 JSON configurations, matching IPv4 destination addresses while forcefully decrementing the network TTL to mimic Layer 3 behavior.
- It iterates through the provided shortest path arrays and assigns the correct physical egress port using a pre-calculated port_mapping dictionary.
- Each generated flow is explicitly saved to a local flows/ directory, creating an organized queue of files ready for controller injection.

```

1  import requests
2  import json
3
4  ODL_IP = "127.0.0.1"
5  AUTH = ('admin', 'admin')
6  HEADERS = {'Content-Type': 'application/json', 'Accept':
7             'application/json'}
8
9  def push_flow_to_odl(node_id, flow_id, json_file_path):
10     url =
11         "http://%s:8181/restconf/config/opendaylight-inventory:nodes/node/openflow:%s" %
12         (ODL_IP, node_id, flow_id)
13
14     with open(json_file_path, 'r') as f:
15         flow_data = json.load(f)
16
17     try:
18         response = requests.put(url, auth=AUTH, headers=HEADERS,
19                                json=flow_data)
20         if response.status_code in [200, 201]:
21             print("Successfully pushed flow %s to openflow:%s" %
22                   (flow_id, node_id))
23         else:
24             print("Failed to push flow. Status: %s. Resp: %s" %
25                   (response.status_code, response.text))
26     except Exception as e:
27         print("RESTCONF connection error: %s" % e)

```

Listing 17: send_flows.py: RESTCONF Data Injector

- This module utilizes the Python requests library to interface with the OpenDaylight RESTCONF application programming interface.
- It formats the exact target URL corresponding to the specific switch (node_id) and flow entry (flow_id) in the operational OpenFlow tree.
- HTTP PUT requests are utilized to send the JSON structures, and standard error handling ensures that any HTTP or connection exceptions are logged to the console.

```

1  import time
2  import os
3  import requests
4  from spf import calculate_spf
5  from create_flows import save_flows
6  from send_flows import push_flow_to_odl
7
8  A = [
9      [0, 2, 3, 4],
10     [2, 0, 0, 1],
11     [3, 0, 0, 0],
12     [1, 1, 0, 0]
13 ]
14 N_NODES = len(A)
15 ODL_IP = "127.0.0.1"
16 AUTH = ('admin', 'admin')
17
18 port_counters = {i+1: 1 for i in range(N_NODES)}
19 port_counters[1] += 1
20 port_counters[N_NODES] += 1
21
22 expected_links = []
23 port_mapping = {}
24
25 for i in range(N_NODES):
26     for j in range(i+1, N_NODES):
27         if A[i][j] > 0 or A[j][i] > 0:
28             src = i + 1
29             dst = j + 1
30             src_port = port_counters[src]
31             dst_port = port_counters[dst]
32
33             expected_links.append((src, dst, src_port, dst_port))
34             port_mapping[(src, dst)] = src_port
35             port_mapping[(dst, src)] = dst_port
36
37             port_counters[src] += 1
38             port_counters[dst] += 1
39
40 def get_inventory():
41     url =
42         "http://%s:8181/restconf/operational/.opendaylight-inventory:nodes/"
43         % ODL_IP
44     try:
45         resp = requests.get(url, auth=AUTH, headers={'Accept':
46             'application/json'})
47         if resp.status_code == 200:
48             return resp.json()
49     except:

```

```

47         pass
48     return None
49
50 def monitor_network():
51     last_path_1_n = None
52     last_path_n_1 = None
53
54     print("Starting bulletproof inventory watcher...")
55
56     while True:
57         inventory = get_inventory()
58         if not inventory or 'nodes' not in inventory:
59             time.sleep(5)
60             continue
61
62         nodes = inventory.get('nodes', {}).get('node', [])
63         if not nodes:
64             time.sleep(5)
65             continue
66
67         port_states = {}
68         for node in nodes:
69             try:
70                 node_id = int(node['id'].split(':')[1], 16)
71                 port_states[node_id] = {}
72                 for nc in node.get('node-connector', []):
73                     port_num = nc.get('flow-node-inventory:port-number')
74                     state = nc.get('flow-node-inventory:state', {})
75                     is_down = state.get('link-down', False)
76                     port_states[node_id][str(port_num)] = is_down
77             except:
78                 continue
79
80         active_links = []
81         for src, dst, src_port, dst_port in expected_links:
82             src_down = port_states.get(src, {}).get(str(src_port), True)
83             dst_down = port_states.get(dst, {}).get(str(dst_port), True)
84
85             if not src_down and not dst_down:
86                 active_links.append((src, dst))
87                 active_links.append((dst, src))
88
89         if not active_links:
90             time.sleep(5)
91             continue
92
93         path_1_n, path_n_1 = calculate_spf(A, active_links, N_NODES)
94
95         if path_1_n and path_n_1 and (path_1_n != last_path_1_n or
96             path_n_1 != last_path_n_1):
97             print("\n=====")
98             print("Topology Change Detected!")
99             print("New Route H1 -> H4:", path_1_n)
100            print("New Route H4 -> H1:", path_n_1)
101            print("Pushing OpenFlow rules...")
102            print("=====\n")
103
104            # CLEAR PREVIOUS FLOWS TO PREVENT CONFLICTS
105            os.system(f"curl -s -u admin:admin -X DELETE
106                http://{ODL_IP}:8181/restconf/config/opendaylight-inventory:nodes
107                > /dev/null")

```

```

105         time.sleep(1) # Wait a second for controller to digest
                        deletion
106
107         flow_files = save_flows(path_1_n, path_n_1, port_mapping)
108         for node, flow_id, filepath in flow_files:
109             push_flow_to_odl(node, flow_id, filepath)
110
111         last_path_1_n = path_1_n
112         last_path_n_1 = path_n_1
113
114         time.sleep(5)
115
116 if __name__ == "__main__":
117     monitor_network()

```

Listing 18: watcher.py: Dynamic Topology Daemon

- This master daemon pre-calculates exact Mininet physical port mappings based on the matrix sequence to reliably interface with raw hardware states.
- It periodically executes GET requests to the opendaylight-inventory endpoint to inspect the explicit link-down boolean status of every registered node connector.
- When physical port statuses change, it updates the active_links array and calls the calculate_spf module to dynamically derive new viable routing paths.
- If a path alteration is detected, the script utilizes a curl DELETE command to securely flush all previous flow rules from the controller, preventing routing loops before injecting the newly calculated survival flows.

4.2 Flow Configurations and Analysis

Our automated Python scripts generated specific JSON payloads designed to enable accurate Layer 3 routing across the asymmetric network paths. An analysis of these exported files confirms proper dynamic routing execution:

- Proper Subnet Targeting & Actions: The active configurations successfully match traffic targeted at the isolated subnets (10.0.2.10/32 and 10.0.1.10/32). Crucially, every active payload strictly enforces the dec-nw-ttl instruction required to simulate a valid Layer 3 routing hop.
- Handling Asymmetric Paths: During stable network operation, the forward route operates across [1, 2, 4] while the reverse route operates across [4, 1]. The scripts efficiently map this asymmetry: Node 4 maintains a forward instruction towards the host and a reverse instruction directly to Node 1, while Node 2 operates exclusively via a single forward flow as it is bypassed entirely on the reverse trip.
- Dynamic Failover Execution: When a link failure necessitates a transition to the [1, 4] fallback route, the Python daemon recalculates and actively pushes an updated configuration to Node 1, successfully altering its output-node-connector to port 4 to seamlessly bypass the severed link.
- Deprecation of Legacy Rules: Prior faulty logic generating flat Layer 2 forwarding actions without TTL decrements (e.g., matching the incorrect 10.0.0.4/32 block) was identified and systematically purged from the active injection directory.

4.3 Results and Command Execution

To validate the automated recovery mechanisms, the environment was manipulated using standard Mininet link control commands. The testing sequence was executed as follows:

1. `sudo python3 create_net.py`: Initiates the virtual network attached to OpenDaylight.
2. `sudo python3 watcher.py`: Starts the inventory monitoring daemon in a secondary terminal.
3. `pingall`: Verifies the initial primary path connectivity.
4. `link s1 s2 down`: Severely disrupts the primary route via the Mininet CLI.
5. `pingall`: Confirms the automatic calculation and instantiation of the backup route.

```
iman@iman-VMware-Virtual-Platform:~$ sudo python3 watcher.py
[sudo] password for iman:
Starting bulletproof inventory watcher...

=====
Topology Change Detected!
New Route H1 -> H4: [1, 2, 4]
New Route H4 -> H1: [4, 1]
Pushing OpenFlow rules...
=====

Successfully pushed flow 1 to openflow:1
Successfully pushed flow 1 to openflow:2
Successfully pushed flow 1 to openflow:4
Successfully pushed flow 2 to openflow:4
Successfully pushed flow 2 to openflow:1

=====
Topology Change Detected!
New Route H1 -> H4: [1, 4]
New Route H4 -> H1: [4, 1]
Pushing OpenFlow rules...
=====

Successfully pushed flow 1 to openflow:1
Successfully pushed flow 1 to openflow:4
Successfully pushed flow 2 to openflow:4
Successfully pushed flow 2 to openflow:1
```

Figure 15: The watcher daemon automatically updating paths following a link failure.

The watcher terminal output explicitly records the life cycle of the dynamic routing adjustments. Initially, the script discovers the topology and pushes flows for the optimal path [1, 2, 4]. Following the manual link disruption, the polling interval immediately triggers a "Topology Change Detected!" alert. The application correctly calculates the secondary path [1, 4], purges the outdated configurations, and successfully pushes the new restorative flows into the operational datastore of the impacted switches.

```
iman@iman-VMware-Virtual-Platform:~$ sudo python3 create_net.py
*** Configuring hosts
h1 h4
*** Starting controller
c1
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h4
h4 -> h1
*** Results: 0% dropped (2/2 received)
mininet> link s1 s2 down
mininet> pingall
*** Ping: testing ping reachability
h1 -> h4
h4 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Figure 16: Mininet CLI confirming uninterrupted connectivity after route degradation.

The Mininet CLI validates the physical outcome of the controller operations. The first `pingall` test yields a 0% drop rate across the primary path [1, 2, 4]. After explicitly forcing the `link s1 s2 down` state, the second `pingall` execution effortlessly succeeds, again reporting 0% dropped (2/2 received). This confirms that the Python daemon successfully insulated the network from physical degradation by seamlessly transitioning data delivery to the backup matrix path.